Appendix A

파이썬 프로그래밍

A.1 파이썬 언어

A.2 데이터 타입과 변수

A.3 연산자와 식

A.4 조건문

A.5 반복문

A.6 컨테이너 객체 다루기

A.7 함수

A.8 모듈

A.9 파일 입출력

■ 연습문제

Appendix A

파이썬 프로그래밍

들어가기 전에

부록 A를 학습하기 전에 두 가지 점을 명확히 하고자 한다. 첫째, 이 책에서 앞으로 다룰 파이썬 프로그래밍을 위해 그리 높지 않은 수준으로 파이썬 언어를 간단히 정리한다. 이 장은 C/C++, Java, 자바스크립트 등 한 가지라도 프로그래밍 언어를 접해본 학생들을 대상으로 기술된다. 아무 프로그래밍 언어도 접해본 적이 없는 학생이 이 장을 읽으면 다소 어렵다고 느낄 수 있다. 둘째, 리눅스에서 파이썬 프로그래밍을 진행한다. 독자들이 윈도우 PC나 맥 PC에서 파이썬 프로그래밍을 연습할 수 있지만, 라즈베리파이에서 이 장의 파이썬 프로그래밍을 학습하기 권한다. 파이썬 프로그램을 작성하고 실행하는 방법은 윈도우, 맥, 라즈베리파이 운영체제(리눅스)에서 차이가 없지만, 라즈베리파이에서 실습하기를 권하는 것은 뒷장에서 다룰 라즈베리파이와 리눅스 환경에 익숙해지기 위함이다. 셋째, 홈 디렉터리에 appendixA 디렉터리를 만들고 그곳에서 코딩을 시작한다.

A.1 파이썬 언어

파이썬(Python)은 널리 사용되는 범용 프로그래밍 언어이다. 1991년 귀도 반 로섬(Guido van Rossum)에 의해 처음 개발된 후 파이썬 소프트웨어 재단(Python Software Foundation)에서 추가로 개발하였다. 파이썬은 코드 가독성에 중점을 두고 설계되었으며 개발자가 더 적은 줄의 코드로 코딩을 할 수 있도록 설계되었다.

현재, 파이썬 프로그램은 Windows, Mac, Linux, Raspberry Pi 등 다양한 플랫폼에서 개발되고 실행될 수 있다. 파이썬 프로그램은 인터프리터 방식으로 실행되므로, 개발자는 컴파일 과정 없이 몇 줄의 코드라도 작성하자마자 바로 실행해 볼 수 있다. 이러한 특징은 초보자가 파이썬 프로그램을 테스트하고 개발하는 것을 쉽게 한다.

파이썬 설치 및 확인

라즈베리파이 운영체제에는 파이썬 3 버전이 이미 포함되어 있어 다른 버전을 설치하지 않는 한 따로 설치할 필요가 없다. 이 책에서는 파이썬 3을 사용한다. 파이썬 3 버전이 설치되었는지 확인하려면 쉘에서 다음과 같이 python 명령을 입력하여 실행 여부를 확인하면 된다.

```
pi@pi:~ $ mkdir appendixA
pi@pi:~ $ cd appendixA
pi@pi:~/appendixA $ python
Python 3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

python 명령의 실행 결과를 보면 Python 3.13.5가 설치되어 있음을 알 수 있다(파이썬 버전은 변할 수 있음). python 프로그램을 종료하려면 단축키 [Ctrl+D]를 입력하면 된다.

잠깐! **Raspberry Pi에서 Pi는 무엇일까?**

Raspberry Pi에서 Pi는 python을 의미한다. 파이썬이 개발될 당시에는 미국의 Apple, 캐나다의 Blackberry, 영국의 Tangerine, Apricot, Acorn 등 컴퓨터 이름에 과일 이름을 붙이는 유행이 있었고, 이런 유행에 따라 Raspberry라는 이름을 붙여졌으며, 프로그래밍 교육에 C/C++보다 쉬운 파이썬 언어로 교육할 것을 주문하기 위해 Pi라는 이름이 추가되었다.

파이썬 프로그램 실행 방식 2가지

파이썬이 설치되면 기본적으로 필요한 패키지를 포함하여 사용자가 입력한 파이썬 코드나 사용자가 작성한 파이썬 프로그램을 실행해주는 python 프로그램이 설치된다.

python 프로그램에는 사용자가 작성한 파이썬 코드를 실행시키는 2가지 방식이 있다.

- 대화식 실행 방식 - 파이썬 코드를 한 개씩 실행시키는 방법
- 파이썬 파일 단위로 실행하는 방식 - 파이썬 프로그램이 담긴 파일을 실행시키는 방법

대화식 실행 방식

대화식 실행 방식이란 사용자로부터 문장 단위로 파이썬 프로그램을 입력받아 실행하는 방식이다. 먼저 python 명령을 입력하여 python 프로그램을 실행시킨다. 그러면 python은 프롬프트 ">>>"를 출력하고 사용자로부터 파이썬 코드의 입력을 기다린다. 사용자가 이곳에 width = 10과 같은 파이썬 코드를 한 라

인 입력하면, 이 라인을 바로 실행시킨다. 다음은 python 프로그램을 실행시킨 후, 4개의 파이썬 코드를 한 줄씩 입력하여 실행시키는 사례이다.

```
pi@pi:~/appendixA $ python
Python 3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> width = 10
>>> height = 20
>>> area = width * height
>>> print("면적은 ", area)
면적은 200
>>>
```

이 방식은 한 번에 한 개의 파이썬 코드는 한 줄에 입력하는 방식으로, 이것은 초보자가 한 줄씩 간단한 파이썬 코드를 연습하거나, 간단한 코드를 테스트하기 위한 목적이다. 파이썬 프로그램이 복잡해지고 길게 되면 대화식 방식의 사용은 거의 불가능하다.

● 파이썬 파일 단위로 실행하는 방식

이 방식은 파이썬 프로그램을 실행시키는 일반적인 방법으로, 파이썬 프로그램을 파일로 저장하고 한 번에 실행시킨다. 이때 파이썬 프로그램 파일의 확장자는 일반적으로 '.py'로 한다.

이제, 이 방식으로 파이썬 프로그램을 실행시켜보자. 독자들은 앞서 대화식 방식에서 실행한 4줄의 파이썬 코드를 vi 편집기를 이용하여 area.py로 저장하라.

```
pi@pi:~/appendixA $ vi area.py
```

```
width = 10
height = 20
area = width * height
print("면적은 ", area)
~
~
~
:wq!
```

이제, 다음과 같이 python 명령으로 area.py를 실행한다.

```
pi@pi:~/appendixA $ python area.py
면적은 200
pi@pi:~/appendixA $
```

초보자가 아닌 이상 대부분 파이썬 프로그램 개발자는 파일 단위로 파이썬 프로그램을 실행시키며, 이 책의 나머지 부분에서도 이 방식을 사용한다.

A.2 데이터 타입과 변수

식별자

식별자(identifier)란 파이썬 프로그램의 변수, 상수(리터럴), 함수에 붙이는 이름이다. 개발자는 식별자를 만들 때 다음 규칙을 따라야 한다.

- 첫 번째 문자 - 알파벳(A-Z, a-z), 언더스코어(_), $ 문자만 사용 가능
- 두 번째 이상 문자 - 알파벳, 언더스코어(_), 0-9, $ 문자 사용 가능
- 대소문자 구분 - myHome과 myhome은 다른 식별자

다음은 식별자로 적합한 것과 그렇지 않은 것에 대한 예이다.

```
6variable       # (X) 숫자로 시작할 수 없음
student_ID      # (○)
_code           # (○) 맞지만 권하지 않음
if              # (X) 예약어 if 사용 불가
%calc           # (X) % 사용 불가
bar, Bar;       # (○) bar와 Bar는 서로 다른 식별자임에 주의
```

잠깐! **파이썬에서 식별자 만드는 관례**

프로그래밍 언어마다 식별자 이름을 만드는 관례는 서로 다르다. 파이썬 언어에서 식별자를 만드는 관례를 알아보자.

(1) 식별자는 의미를 담을 수 있도록 최대한 길게 쓰는 것이 좋다. 의미를 잘 담을 수 있게 붙여진 좋은 이름은 프로그램에 가독성을 높여 주기 때문이다. 예를 들면 다음과 같다.

```
number, next_node, number_of_students
```

(2) 두 개 이상의 단어로 구성되는 식별자를 만들 때 '_'로 연결한다. 사례는 다음과 같다.

```
number_of_files, student_ID, social_security_number
```

식별자의 이름 붙이는 관례는 매우 중요하다. 이름을 보고 변수인지 상수인지, 클래스 이름인지 단번에 알 수 있으며, 개발자 상호 간에 의사소통에도 매우 중요하다.

주석문

주석문은 파이썬 프로그램 내에 설명이나 메모 목적으로 삽입한 텍스트로서, 프로그램 코드로 처리되지 않는다. 주석문은 다음과 같이 한 라인 주석문과 여러 라인 주석문의 2가지 형태가 있다.

- 한 라인 주석문 - # 으로 시작하여 그 줄 끝까지 주석 처리
- 여러 라인 주석문 - """와 """ 사이에 여러 라인을 주석 처리

앞 사례의 area.py 코드에 주석문을 붙이면 다음과 같이 작성할 수 있다.

```
"""
이것은 처음 작성한 파이썬 프로그램입니다.
작성자 : 황기태, 이재문
작성일 : 4월 14일
"""
width = 10                  # 너비 10
height = 20                 # 폭 20
area = width * height       # 면적 계산
print("면적은 ", area)      # 면적 출력
```

```
면적은 200
```

문장 구분

파이썬 프로그램의 기본 단위는 문장(statement)이며, 세미콜론(;)으로 문장과 문장을 구분한다. 한 줄에 한 문장만 있는 경우 세미콜론을 생략할 수 있다. 다음은 문장 구분의 사례이며, 마지막 문장은 오류 문장이다.

```
i = i + 1             # (○) 한 줄에 한 문장이므로 세미콜론 생략
j = j + 1;            # (○)
k = k + 1; m = m + 1; # (○) 한 줄에 여러 문장
n = n + 1 p = p + 1;  # (X) 첫 번째 문장(n = n + 1) 끝에 세미콜론(;) 필요
```

화면 출력

프로그램을 작성할 때 가장 먼저 알아야 하는 것이 화면에 출력하는 방법과 사용자로부터 키 입력을 받는 방법이다. 지금부터 파이썬 언어에서 화면에 출력하고 키 입력받는 방법에 대해 알아보자.

● print()를 이용한 화면 출력

print() 함수의 매개변수를 이용하여 데이터나 텍스트를 화면에 출력한다. 예를 들어보자.

■ 1개의 정보를 출력하는 사례

```
01 print(3)
02 print(2.5)
03 print(2+3)
04 print("hello")
```

```
3
2.5
5
hello
```

■ 2개 이상의 정보를 출력하려면 다음과 같이 콤마(,) 사용

```
01 print(3, 2.5) # 32.5 출력
02 print("파이썬 성적은 ", 90, "점입니다")
```

```
32.5
파이썬 성적은 90점입니다
```

■ 변수에 저장된 값을 출력하는 사례

```
01 score = 90
02 text = "파이썬 성적은 "
03 print(text, score, "점입니다")
```

```
파이썬 성적은 90점입니다
```

● print() 함수의 end=" " 옵션 활용

print() 함수는 자동으로 개행 문자(\n)를 출력하여 다음 줄로 넘어간다. 다음은 두 줄에 걸쳐 출력된다.

```
01 print("2 + 3은")
02 print(2+3, "입니다.")
```

```
2 + 3은
5입니다.
```

end=" " 옵션을 이용하면, 개행 문자(\n) 대신, 빈 문자를 출력할 수 있다. 다음은 두 print() 출력이 한 줄에 연이어 출력한다.

```
01 print("2 + 3은", end=" ")
02 print(2+3, "입니다.")
```

```
2 + 3은 5입니다.
```

첫 번째 print()는 "2 + 3은"을 출력한 후 빈 문자(" ")를 출력하고, 두 번째 print()는 "5입니다"를 출력한다.

다음은 3개의 print() 출력이 "; "와 함께 한 줄에 연이어 출력되며, 마지막 출력 후에도 커서는 다음 줄로 넘어가지 않는다.

```
01 print("C++", end="; ")
02 print("Java", end="; ")
03 print("Python", "; ")
```

```
C++; Java; Python;
```

● 포맷 출력

print()로 정보를 출력할 때 다음 포맷 문자를 이용하여 포맷을 지정할 수 있다.

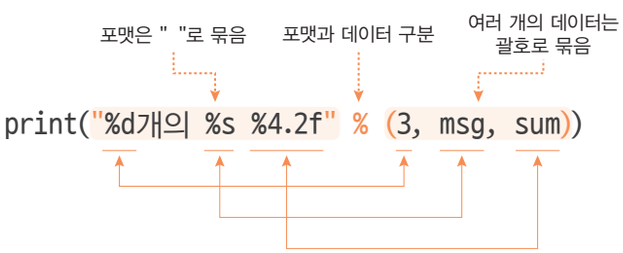
```
포맷 문자 %s, %d, %f 활용
```

%s는 문자열을 출력할 때, %d는 정수를 출력할 때, %f는 실수를 출력할 때 사용한다. 포맷은 " "로 묶고, 포맷에 대응하는 값이나 변수들은 ( )로 묶고 이들 사이에는 %로 분리한다. 예를 들어보자.

```
01 msg = "점수 합은"
02 sum = 98.7
03 print("%d개의 %s %4.2f" % (3, msg, sum))
```

```
3개의 점수 합은 98.70
```

이 사례는 [그림 A-1]과 같이 %d 포맷은 숫자 3과 대응하고, %s는 msg 변수와 대응하며, %4.2f는 sum 변수에 대응한다. %4.2f는 sum 변수에 저장된 값을 4개의 유효숫자로 출력하되 소수점 이하를 2자리로 출력할 것을 지시하는 것이다.



[그림 A-1] print()의 포맷과 데이터와의 관계

키 입력

파이썬 언어에서 사용자로부터 키 입력을 받을 때 input() 함수를 사용한다. input() 함수는 매개변수로부터 전달받은 문자열을 화면에 출력하고 사용자의 입력을 기다린 후 사용자의 입력을 문자열 형태로 리턴한다. 그러므로 정수를 입력받고자 하면 input()이 리턴한 문자열을 정수로 바꾸는 int() 함수를, 실수를 입력받고자 하면 input()이 리턴한 문자열을 실수로 바꾸는 float() 함수를 사용해야 한다. 〈표 A-2〉를

참고하라.

input()으로 키 입력을 받는 몇 가지 사례를 보자.

■ **사용자로부터 이름을 입력받는 사례**

```
name = input("이름을 입력하세요>> ") # 사용자가 입력한 문자열이 name 변수에 저장
```

■ **정수를 입력받는 사례**

```
age = int(input("나이를 입력하세요>> ")) # 사용자 입력을 정수로 바꾸어 age에 저장
```

■ **실수를 입력받는 사례**

```
weight= float(input("몸무게를 입력하세요>> ")) # 사용자 입력을 실수로 바꾸어 weight에 저장
```

예제 A-1 이름과 점수를 입력받아 평균을 출력하는 사례

이름과 과목 2개의 점수를 입력받아 이름과 평균을 출력하라.

A-1.py

```
01 name = input("이름을 입력하세요>> ")
02 eng = int(input("영어 성적을 입력하세요>> "))
03 history  = int(input("역사 성적을 입력하세요>> "))
04 print(name, "님은 성적 평균은 ", (eng+history)/2, "입니다.")
```

```
pi@pi:~/appendixA $ python A-1.py
이름을 입력하세요>> 이재문
영어 성적을 입력하세요>> 80
역사 성적을 입력하세요>> 90
이재문 님은 성적 평균은 85.0입니다.
pi@pi:~/appendixA $
```

데이터 타입

데이터 타입(data type)이란 파이썬 언어로 다룰 수 있는 데이터의 종류이다. 정육사가 다룰 수 있는 데이터는 돼지고기, 소고기, 닭고기, 양고기로 볼 수 있고, 일식 요리사가 다룰 수 있는 데이터는 숭어, 복어, 광어, 전어 등의 생선류가 될 것이다. 그러면 파이썬 언어로 다룰 수 있는 데이터의 종류에는 무엇이 있을까?

● 데이터 타입 종류

파이썬 언어의 데이터 타입은 다른 컴퓨터 언어에 비교하여 단순하다. 다음은 파이썬 언어에서 데이터 타입과 값에 대한 예이다.

- 숫자 타입 : 정수, 실수(예: 42, 3.14)
- 논리 타입 : 참과 거짓을 표현하는 타입으로, True, False (true, false는 아니므로 주의할 것)
- 문자열 타입 : "좋은 세상", "a", "365", "2+4"
- 객체 레퍼런스 타입 : 객체를 가리킴. C 언어의 포인터와 유사
- None : 값이 없음을 표시하는 특수 키워드, 다른 언어의 NULL, null과 유사

논리 타입에서 True와 False는 참과 거짓을 나타내는 파이썬 예약어이다. 파이썬에서는 문자 타입이 없어 문자열로 표현한다. 파이썬에서 객체 레퍼런스는 객체의 주소를 나타내는 타입으로, 자바의 레퍼런스와 의미가 유사하다.

데이터 타입 및 변수 선언

변수는 파이썬 프로그램이 실행 중에 데이터를 저장하는 공간이며 반드시 이름을 가진다. 프로그램에서 변수를 만드는 작업을 변수 선언이라고 부르는데, '변수=값'의 방식으로 매우 간단히 변수를 선언한다. 이때 중요한 것은 이미 선언되지 않은 새 이름의 변수를 사용할 때 변수가 선언된다는 점이다.

다음은 age와 score 변수를 선언하는 사례이다.

```
age = 21        # 변수 age를 선언하고 21로 초기화
score = 10      # 변수 score를 선언하고 10으로 초기화
```

이 코드는 age라는 이름의 변수를 선언하고 초깃값을 21로 저장하고, score라는 이름의 변수를 선언하고 초깃값으로 10을 저장하였다. 변수를 선언하였으면 다음과 같이 사용할 수 있다.

```
age = age + 1       # age에 저장된 값을 읽어서 1 증가한 후 다시 저장
score = 90          # score에 90 저장
```

파이썬에서는 변수에 특정 데이터 타입의 값만 저장할 수 있는 것은 아니다. 다음과 같이 어떤 값을 저장해도 상관없다.

```
score = 66.8        # 실수도 저장 가능
score = "high"      # 문자열도 저장 가능
```

리터럴

변수가 데이터 저장 공간의 이름이라면, 리터럴(literal)은 값 그 자체이다. 다음은 변수 x에 리터럴 10을 저장하는 코드이다.

```
x = 10      # x는 변수, 10은 리터럴
```

리터럴의 종류는 〈표 A-1〉과 같다. 문자열 리터럴은 이중 인용부호("")나 단일 인용부호('') 중 아무것이나 사용해도 된다. 문자열 내에 이중 인용부호 문자(")나 단일 단일인용부호 문자(')를 사용하고자 하면, 백슬레시 문자(\)를 사용하여 \"나 \'로 사용하면 된다. 다음은 문자열 내 이중 인용부호 문자(")를 포함하는 2가지 경우이다.

```
print("그녀는 \"누구세요\"라고 했습니다.")        # \를 사용해야 함
print('그녀는 "누구세요"라고 했습니다.')          # \를 사용하지 않아도 됨
```

〈표 A-1〉 리터럴의 종류와 예

구분		특징	예
정수	8진수	0o로 시작(숫자 0과 문자 o)	n = 0o15 # 8진수, 10진수로 13
	10진수		n = 15 # 10진수 15
	16진수	0x로 시작(숫자 0과 문자 x)	n = 0x15 # 16진수 15, 10진수로 21
실수	소수형		height = 0.12345
	지수형		height = 12345E-4 # 1234*10-4
논리	참	True	condition = True # 참
	거짓	False	condition = False # 거짓
문자열		" "로 묶음	hello = "안녕하세요"
		' '로 묶음	name = 'kitae'
기타	None	값이 없음을 뜻함	ret = None # ret에 저장된 값이 없음

예제 A-2 리터럴 보기

파이썬의 다양한 리터럴 사용 예이다.

A-2.py

```
01 oct = 0o15          # 0o15는 8진수 15 -> 10진수로 13
02 hex = 0x15          # 0x14는 16진수 15 -> 10진수로 21
03 condition = True    # 소문자 true로 하면 안 됨
04
05 print("8진수 0o15는 십진수로 %d" % oct)
06 print("16진수 0x15는 십진수로 %d" % hex)
07
08 print("condition은 " + str(condition))
09
10 print('문자열 : 단일인용부호로도 표현')
11 print("그녀는 \"누구세요\"라고 했습니다.")
12 print('그녀는 "누구세요"라고 했습니다.')
```

```
pi@pi:~/appendixA $ python A-2.py
8진수 0o15는 십진수로 13
16진수 0x15는 십진수로 21
condition은 True
문자열 : 단일인용부호로도 표현
그녀는 "누구세요"라고 했습니다.
그녀는 "누구세요"라고 했습니다.
pi@pi:~/appendixA $
```

상수

일반적으로 상수는 프로그램 실행 동안 고정된 값을 가지는 이름을 뜻한다. 대부분 언어에서는 상수를 선언하는 별도의 방법을 지원하지만, 파이썬에서 상수를 지원하지 않기 때문에 개발자 스스로 변수에 초깃값을 저장한 이후 값을 바꾸지 않는 방식으로 상수를 구현한다.

일반적으로 상수를 변수와 구분하기 위해 상수 이름을 대문자로만 붙이도록 권한다. 예를 들면 다음과 같다.

```
PI = 3.14
GRAVITY = 9.8
UNIT = 1000

area = PI*3*3      # 반지름이 3인 원의 면적
```

다음과 같이 상수에 값을 변경한다고 하더라도 프로그램은 오류를 발생시키지 않는다. 파이썬에서는 상수를 변수와 별도로 다루지 않기 때문이다.

```
PI = 3.141592      # 앞에서 3.14로 저장된 PI 변수의 값을 변경하는 것이 가능
```

데이터형 변환

프로그램에서 종종 데이터의 형을 변환하여야 하는 경우가 발생한다. 예를 들어 문자열 "10"과 숫자 20을 더하려고 하면 문자열 "10"을 숫자 10으로 변환해서 20과 더해야 한다. 파이썬은 〈표 A-2〉와 같은 여러 변환 함수를 제공한다. 간단한 사례를 알아보자.

```
total = int("10") + 20          # 문자열 "10"을 10으로 변환하고 20을 더해 30을 total에 저장
msg = "가격은 " + str(total)    # total에 저장된 30을 "30"으로 변환하여, msg 변수에 "가격은 30"이 저장
```

다음은 형 변환 중에 오류가 발생하는 경우이다.

오류

```
m = int("10.5")    # 문자열 "10.5"를 정수로 바꿀 수 없어 오류 발생
n = int(10.5)      # 실수 10.5를 10으로 바꿀 수 있어 오류 아님
```

〈표 A-2〉 데이터형 변환 함수들

함수명	내용
int(value)	value를 정수로 변환하여 리턴, 변환 불가능하면 오류 발생하고 프로그램 종료
float(value)	value를 실수로 변환하여 리턴, 변환 불가능하면 오류 발생하고 프로그램 종료
str(value)	value를 문자열로 변환하여 리턴
chr(value)	value를 문자열로 변환하여 리턴, value가 정수가 아니면 오류 발생하고 프로그램 종료
bool(value)	value가 0이면 False, 그렇지 않으면 무조건 True로 하여 변환하여 리턴
ord(value)	value가 1개의 문자로만 구성된 문자열이면, 문자에 대한 ASCII 코드 리턴

예제 A-3 형 변환 함수 활용

파이썬의 다양한 형 변환 함수를 사용하는 방법을 보인다.

A-3.py

```
ten = int("10")          # ten = 10과 같은 결과
twenty = int(20.5)       # twenty = 20과 같은 결과
text1 = str(30)          # text1 = "30"과 같은 결과
text2 = str(40.5)        # text2 = "40.5"
a = chr(65)              # a = "A"와 같은 결과. 65는 문자 A에 대한 ASCII 코드
b = ord("A")             # b = 65와 같은 결과. 문자 A의 ASCII 코드는 65
c = bool(0)              # c = False과 같은 결과
d = bool("True")         # d = True와 같은 결과

print("ten=%d, twenty=%d" % (ten, twenty))
print("text1=%s, text2=%s" % (text1, text2))
print("a=%s, b=%d, c=%d, d=%d" % (a, b, c, d))
```

```
pi@pi:~/appendixA $ python A-3.py
ten=10, twenty=20
text1=30, text2=40.5
a=A, b=65, c=0, d=1
pi@pi:~/appendixA $
```

A.3 연산자와 식

파이썬 언어는 연산을 위한 다양한 연산자(operator)를 제공한다. 〈표 A-3〉은 파이썬의 연산과 연산자를 보여준다.

〈표 A-3〉 파이썬 연산과 연산자 종류

연산	연산자
산술	+, -, *, /, //, %
비교	>, >=, <, <=, ==, !=
비트	&, |, ^, ~, <<, >>

연산	연산자
대입	=, +=, -=, *=, /=, //=, &=, |=, ^=, <<=, >>=
논리	and, or, not
조건	if ... else

산술 연산

산술 연산자는 더하기(+), 빼기(-), 곱하기(*), 실수 나누기(/), 정수 나누기(//), 나머지(%)의 6가지이며 연산자의 사용 사례는 다음과 같다.

```
x = 32;
total = 100 + x*2/4 -3;    # total은 113
```

파이썬은 정수 나누기와 실수 나누기를 구분한다. 실수 나누기(/)는 결과는 항상 실수이며, 정수 나누기(//)는 소수 부분을 버리고 정수 부분만 결과가 된다.

```
x = 11/4;      # x는 2.75
y = 11//4;     # y는 2
```

나머지(%) 연산자는 나머지가 정수이든 실수이든 모두 결과가 된다.

```
x = 11%4;      # x는 3
y = 11%4.5     # y는 2.0
```

예제 A-4 산술 연산

다양한 산술 연산 예이다.

A-4.py

```
01 x=32
02 total = 100 + x*2/4 - 3 # total은 113
03 div = x / 10            # div는 3.2
04 divint = x // 10        # divint = 3
05 mod = x % 2             # x를 2로 나눈 나머지, 0
06
07 print("x = %d" % x)
08 print("total = %d" % total)
09 print("div = %f" % div)
10 print("divint = %d" % divint)
11 print("mod = %d" % mod)
```

```
pi@pi:~/appendixA $ python A-4.py
x = 32
total = 113
div = 3.200000
divint = 3
mod = 0
pi@pi:~/appendixA $
```

대입 연산

대입 연산자는 오른쪽 식의 결과를 왼쪽에 있는 변수에 대입하는 것으로 종류는 〈표 A-4〉와 같고 사용 사례는 다음과 같다.

```
a = 1, b = 3
a = b       # a에 b의 값을 대입하여 a = 3, b = 3이 된다.
a += b      # a = a + b의 연산이 이루어져, a = 6, b = 3이 된다.
```

〈표 A-4〉 대입 연산자

연산자	내용	연산자	내용
a = b	b값을 a에 대입	a %= b	a = a % b와 동일
a += b	a = a + b와 동일	a &= b	a = a & b와 동일
a -= b	a = a - b와 동일	a |= b	a = a | b와 동일
a *= b	a = a * b와 동일	a ^= b	a = a ^ b와 동일
a /= b	a = a / b와 동일	a <<= b	a = a << b와 동일
a //= b	a = a // b와 동일	a >>= b	a = a >> b와 동일

예제 A-5 대입 연산

다양한 대입 연산 예이다.

A-5.py

```
01 x=3; y=3; z=3
02 print("x= %d y= %d z=%d" % (x, y, z))
03
04 x += 3  # x=x+3 -> x=6
05 y *= 3  # y=y*3 -> y=9
06 z %= 2  # z=z%2 -> z=1
07
08 print("x += 3; 실행 후, x= %d" % x)
09 print("y *= 3; 실행 후, y= %d" % y)
10 print("z %%= 2; 실행 후, z= %d" % z)
```

```
pi@pi:~/appendixA $ python A-5.py
x= 3 y= 3 z=3
x += 3; 실행 후, x= 6
y *= 3; 실행 후, y= 9
z %= 2; 실행 후, z= 1
pi@pi:~/appendixA $
```

비교 연산

비교 연산은 두 값을 비교하여 결과가 True나 False로 나타내는 연산으로 〈표 A-5〉와 같이 여러 종류가 있다. 비교 연산의 예를 들면 다음과 같다.

```
age = 25
result = (age > 20)        # age가 20보다 크므로 result는 True
```

〈표 A-5〉 비교 연산자

연산자	내용	연산자	내용
a < b	a가 b보다 작으면 True	a >= b	a가 b보다 크거나 같으면 True
a > b	a가 b보다 크면 True	a == b	a가 b와 같으면 True
a <= b	a가 b보다 작거나 같으면 True	a != b	a가 b와 같지 않으면 True

예제 A-6 비교 연산

다양한 비교 연산 예이다.

A-6.py

```
01 x=13; y=7
02 print("x = %d, y = %d" % (x, y))
03 print("x == y : " + str(x == y))
04 print("x != y : " + str(x != y))
05 print("x >= y : " + str(x >= y))
06 print("x > y : " + str(x > y))
07 print("x <= y : " + str(x <= y))
08 print("x < y : " + str(x < y))
```

```
pi@pi:~/appendixA $ python A-6.py
x = 13, y = 7
x == y : False
x != y : True
x >= y : True
x > y : True
x <= y : False
x < y : False
pi@pi:~/appendixA $
```

논리 연산

파이썬의 논리 연산은 〈표 A-6〉과 같이 and, or, not의 3가지이다. and는 피연산자 모두 True이면 True이고 그렇지 않으면 False이다. or는 피연산자 중 하나라도 True이면 True이며, not는 피연산자가 True이면 False이다. 다음은 논리 연산의 사례이다.

```
score = 90
age = 20
res = score > 80 and age < 25   # res에 True를 저장
```

〈표 A-6〉 논리 연산자

연산자	내용
a and b	a, b 모두 True이면 True, 그렇지 않으면 False
a or b	a, b 중 하나라도 True이면 True, 둘 다 False이면 False
not a	a가 True이면 False, False이면 True

예제 A-7 논리 연산

논리 연산의 다양한 예이다.

A-7.py

```
01 x=True; y=False
02 print("x = %s, y = %s" % (str(x), str(y)));
03 print("x and y : "+ str(x and y));
04 print("x or y : "+ str(x or y));
05 print("not x : " + str(not x));
06 print("");
07 print("(3>2) and (3<4) : " + str((3>2)and(3<4)));
08 print("(3==-2) or (-1<0) : " + str((3==2)or(-1<0)))
```

```
pi@pi:~/appendixA $ python A-7.py
x = True, y = False
x and y : False
x or y : True
not x : False

(3>2) and (3<4) : True
(3==-2) or (-1<0) : True
pi@pi:~/appendixA $
```

조건 연산

조건 연산은 다음과 같이 조건식 condition과 2개의 식 expT와 expF로 구성된다.

```
res = expT if condition else expF
```

condition이 True이면 조건식의 전체 결과는 expT의 계산 값이 res에 저장되고, False이면 expF의 계산 값이 res에 저장된다. 사례를 보자.

```
x=5; y=3
big = x if x > y else y  # x > y가 True이므로 x 값 5가 big에 대입된다.
```

예제 A-8 조건 연산

조건 연산 예이다.

A-8.py

```
01 a=3; b=5
02 print("a = %d, b = %d" % (a, b))
03 print("두수의 차이 : %d" % (a-b if a>b else b-a))
```

```
pi@pi:~/appendixA $ python A-8.py
a = 3, b = 5
두수의 차이 : 2
pi@pi:~/appendixA $
```

문자열 연산

+, += 연산자는 문자열 연산에도 사용된다. 이때 피연산자들은 반드시 문자열이어야 한다. + 연산자로 문자열을 연결하는 사례를 들어보자.

```
"abc" + "de"       # "abcde"
"abc" + str(23)    # "abc23". "abc" + 23의 연산은 오류
str(23) + "abc"    # "23abc"
str(23) + "35"     # "2335"
23 + 35            # 58, 정수 더하기
```

오류

문자열 비교

비교 연산자(>, <, >=, <=, ==, !=)는 문자열 비교에도 사용되며, 사전에서 뒤에 나오는 문자열이 큰 것으로 판단된다. 간단한 사례를 보자.

```
name = "kitae"
res = name == "kitae"  # == 연산의 결과는 True
res = name > "park"    # name이 "park"보다 사전 순으로 앞에 나오므로 결과는 False
```

예제 A-9 문자열 연산

문자열의 비교 연산 예이다.

A-9.py

```
01 print("abc" + str(23))
02 print(str(23) + "abc")
03 print(str(23) + "35")
04 print(str(23 + 35))
05 print(str(23 + 35) + "abc")
06 print("abc" + str(23 + 35))
07
08 name = "kitae"
09 print(name == "kitae")
10 print(name > "park")
```

```
pi@pi:~/appendixA $ python A-9.py
abc23
23abc
2335
58
58abc
abc58
True
False
pi@pi:~/appendixA $
```

A.4 조건문

조건문은 조건을 검사하고 그 결과가 True인지 False인지에 따라 서로 따른 작업(프로그램 코드)을 실행하는 문장이다. 파이썬 조건문은 if, if-else, if-elif-else의 3종류가 있다.

if

if 문만으로 조건문을 만들 수 있으며 모양과 활용 사례는 다음과 같다.

```
if 조건식 :
   ... 실행문 ... # 조건식이 True이면 실행

if a > b :
   print("a가 b보다 큽니다.")
```

if 문의 조건식이 True이면 실행문을 실행하며, False이면 실행문을 실행하지 않는다.

if-else

if-else 문을 이용하면 조건식이 True인 경우와 False인 경우, 각각 다른 코드를 실행할 수 있다.

```
if 조건식 :
   ... 실행문1 ... # 조건식이 True인 경우 실행
else:
   ... 실행문2 ... # 조건식이 False인 경우 실행
```

```
if a > 10 :
   print("%d가 10보다 큽니다." % a)
else:
   print("%d가 10보다 작지 않습니다." % a)
```

if-elif-else

if-else 문을 연속 작성하여 각 조건에 맞는 코드를 실행하고자 할 때 if-elif-else 문을 사용하면 된다.

```
if 조건식1 :
   ... 실행문1 ...  # 조건식1이 True인 경우
elif 조건식2 :
   ... 실행문2 ...  # 조건식1이 False이고, 조건식2가 True인 경우 실행

… … … …
else:
   ... 실행문n ...  # 앞의 모든 조건이 False인 경우 실행
```

```
if a > b :
   print("큰 수는 %d" % a);
elif a < b :
   print("큰 수는 %d" % b);
else:
   print("%d와 %d는 같다." % (a, b));
```

블록과 들여쓰기 – 중요하니 잘 알아두자.

올 것이 왔다. 파이썬 초보자의 코딩에서 많은 오류가 발생하는데, 지금 다루려고 하는 블록에 대한 들여쓰기이다. 함수나, if, for, while 등에서 조건을 만족할 때 실행할 문장들을 묶어 블록(block)이라고 한다. C/C++, Java, C#, 자바스크립트 등 대부분 컴퓨터 언어들은 블록 개념을 두고 있으며, 많은 경우 '{'

와 '}' 기호로 코드를 묶어 하나의 블록을 표현한다. '{'는 블록의 시작 기호이고 '}'는 블록의 종료 기호이다.

하지만, 다른 언어들과 달리, 파이썬 언어에서는 블록을 나타내기 위해 시작 기호로 ':'를 사용하고 종료 기호가 없다. 대신에 블록에 속한 코드들을 들여쓰기(indentation)하는 식으로 블록을 표현하여 블록 종료를 알 수 있도록 한다. 그러므로 한 블록 내의 코드들은 반드시 동일한 들여쓰기가 이루어져야 한다. 들여쓰기는 빈칸으로 개수를 맞추든지, 탭 키를 입력하는 방식으로 하면 된다.

들여쓰기 예를 들면 다음과 같다.

```
if temperature > 30 :          ← 블록의 시작
tab print("온도가 30이상.")
tab print("경보 발송!")
tab print("온도는 낮추도록 에어컨을 작동시켜라.")   ← 들여쓰기 일치
else :                         ← 블록의 시작
tab print("온도가 30도 이하.")
```

if 문 끝에 콜론(:)을 표기하여 블록의 시작임을 표시하고, 다음 라인부터는 블록을 나타내기 위해 들여쓰기를 시행한다. 앞의 if 문은 3개의 문장이 하나의 블록을 구성하고 있다.

독자들은 파이썬 코드를 작성할 때 종종 'IndentationError'를 만나게 된다. 예를 들어 print("온도가 30이상.") 문의 앞에 들어간 들여쓰기는 탭(tab) 문자이고 print("경보 발송!") 문의 앞에 들어간 들여쓰기는 8개의 공백 문자라고 하면 사람의 눈으로는 같은 들여쓰기로 보이나 다른 들여쓰기로 여겨 오류를 발생한다. 'IndentationError' 오류가 발생하면 오류가 발생한 라인에 들여쓰기가 정확한지 검사해봐야 할 것이다.

예제 A-10 if-elif-else와 들여쓰기 연습

if-elif-else 문을 활용하여 점수를 입력받아 학점의 계산하여 출력하라.

A-10.py

```
01 grade = 0
02 score = input("점수를 입력하세요>>")
03 score = int(score) # 문자열을 숫자로 바꿈
04
05 if score >= 90 : # score가 90 이상
06    grade = "A"
07 elif score >= 80 : # 80 이상 90 미만
08    grade = "B"
09 elif score >= 70 : # 70 이상 80 미만
10    grade = "C"
11 elif score >= 60 : # 60 이상 70 미만
```

```
12      grade = "D"
13  else: # 60 미만
14      grade = "F"
15  print(str(score) + "는 " + grade + "입니다.")
```

```
pi@pi:~/appendixA $ python A-10.py
점수를 입력하세요>>89
89는 B입니다.
pi@pi:~/appendixA $
```

잠깐! **switch 문은 파이썬에 없다.**

일반적으로 다른 언어에 있는 switch 조건문이 파이썬에는 없다. 그것은 switch 문으로 작성한 코드는 if-elif-else 문으로도 항상 작성하는 것이 가능하기 때문이다.

A.5 반복문

파이썬은 동일한 작업을 반복할 수 있는 다음 2개의 반복문만 두고 있다.

```
for 문, while 문
```

for 반복문

for 문은 반복 회수가 정해져 있는 경우에 주로 사용한다. for 문은 시작 값에서 종료 값까지 반복하는 단순 반복 외에도 뒤에서 다룰 리스트에 저장된 요소들에 대하여 반복에도 사용된다. for 문의 반복되는 코드(문장)들은 들여쓰기를 이용하여 블록으로 처리해야 한다.

● range() 함수와 for 반복문

다음은 range() 함수를 사용하여 시작 값에서 종료 값까지 증가 혹은 감소시켜 가면서 반복하는 for 문의 전형적인 모습이다. 반복 시마다 range()가 리턴하는 값은 '변수'에 저장되어 블록이 실행된다.

```
for 변수 in range(start, stop, step):
    반복될 문장1
    ...
    반복될 문장n
```

여기서 range() 함수의 작동에 대해 알아보자.

- start : 시작 값이며 생략하면 0이 된다.

- stop : 반복 중지 조건 값이며 생략될 수 없다.
- step : 반복 시마다 증가시키는 값으로 생략하면 1이 된다. 음수이면 감소이다.

range() 함수는 start에서 시작하여 stop 앞에 이르기까지 step만큼 증가시키면서 정수를 리턴한다. step이 양수이면 리턴하는 수가 증가하고, step이 음수이면 리턴하는 수는 계속 감소한다. range() 함수는 step을 증가시키면서 stop의 조건을 검사하여 범위 내에 있으면 step을 리턴하고 그렇지 않으면 for 문을 탈출하게 한다. 다음은 range() 함수 사용 예이다.

```
range(5)          # range(0,5,1)과 동일, 0, 1, 2, 3, 4를 순서대로 리턴
range(2, 5)       # range(2,5,1)과 동일, 2, 3, 4를 순서대로 리턴
range(2, 5, 2)    # 2, 4를 순서대로 리턴
range(5, 0, -1)   # 5, 4, 3, 2, 1을 순서대로 리턴
```

다음은 생략된 step이 1이므로 증가하는 방향인데, start가 stop보다 크기 때문에, 아무 값도 리턴하지 않고 반복을 종료하는 경우이다.

```
range(5, 2)       # range(5, 2, 1)으로 아무 값도 리턴하지 않고 반복 종료
```

예제 A-11 range() 함수와 for 반복문

다양한 range() 함수를 활용한 for 문의 예이다.

A-11.py

```
01 # 10에서 35까지 5씩 i 증가
02 for i in range(10, 36, 5):
03    print(i, end=",")
04 print("")
05
06 # i를 0에서 5까지 1씩 증가
07 for i in range(6):
08    print(10+i*5, end=",")
09 print("")
10
11 # i를 0에서 5까지 1씩 증가
12 for i in range(0, 6):
13    print(10+i*5, end=",")
14 print("")
15
16 # i를 35에서 10까지 5씩 감소
```

```
for i in range(35, 9, -5):
   print(i, end=",")
print("")
```

```
pi@pi:~/appendixA $ python A-11.py
10,15,20,25,30,35,
10,15,20,25,30,35,
10,15,20,25,30,35,
35,30,25,20,15,10,
pi@pi:~/appendixA $
```

● 문자열과 for 반복문

for 문은 다음과 같이 "문자열"의 모든 문자에 대해서 순서대로 블록을 반복 실행할 수 있다.

```
for 변수 in 문자열:
   반복될 문장1
   ...
   반복될 문장n
```

예를 들어 "Falling in love with you"의 각 문자를 모두 출력하는 코드는 다음과 같다.

```
for c in "Falling in love with you" :
   print(c, end=",")
```

```
F,a,l,l,i,n,g, ,i,n, ,l,o,v,e, ,w,i,t,h, ,y,o,u,
```

예제 A-12 문자열과 for 반복문

"Falling in love with you"에서 문자 "i"의 개수를 세는 코드를 작성하라.

A-12.py

```
count = 0
for c in "Falling in love with you" :
   if c == "i" :
      count += 1
print("i의 개수는 ", count)
```

```
pi@pi:~/appendixA $ python A-12.py
i의 개수는  3
pi@pi:~/appendixA $
```

while 반복문

while 문은 조건식이 True인 동안 블록을 반복 실행한다. 조건식이 False가 되면 블록을 실행하지 않고 while 문을 탈출한다.

```
while 조건식 :
    반복될 문장1
    ...
    반복될 문장N
```

예제 A-13 while 문

while 문을 활용하여 정수를 입력받아 0에서 입력받은 정수까지의 합을 구하여 출력하는 프로그램을 작성하라.

A-13.py

```
01 n = input("정수 입력>> ")
02 n = int(n) # 입력받은 문자열을 정수로 변환
03 i = 1
04 sum = 0
05 while i <= n :
06     sum += i
07     i += 1
08 print("1~%d까지의 합은 %d" % (n, sum))
```

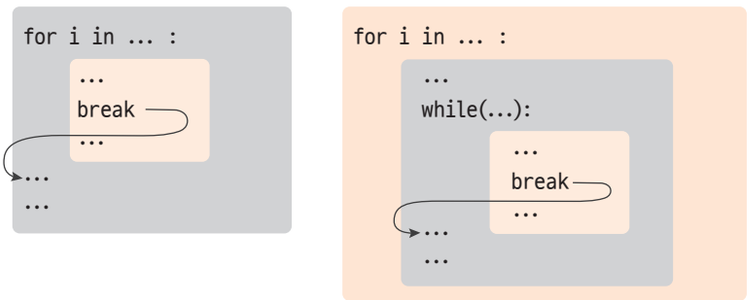
```
pi@pi:~/appendixA $ python A-13.py
정수를 입력하세요: 10
1~10까지의 합은 55입니다.
pi@pi:~/appendixA $
```

잠깐! do-while 반복문은 파이썬에 없다.

일반적으로 다른 언어에 있는 do-while 반복문은 파이썬에 없다. 그것은 for나 while 문으로 얼마든지 대체할 수 있기 때문이다.

반복문 내의 break 문

break 문은 반복문을 벗어나게 한다. 여러 반복문이 중첩된 경우 break 문을 포함하는 한 개의 반복문만 벗어난다. [그림 A-2]는 break 문에 의하여 반복문의 제어가 변경되는 것을 보여준다.



[그림 A-2] break 문으로 반복문을 벗어나는 경우

예제 A-14 break 문

break 문을 활용하여 반복문을 벗어나는 사례를 보인다. 1에서 얼마까지 더하면 3000보다 크면서 3000에 가장 가까운 합을 얻을 수 있는지 프로그램을 작성하여 찾아라.

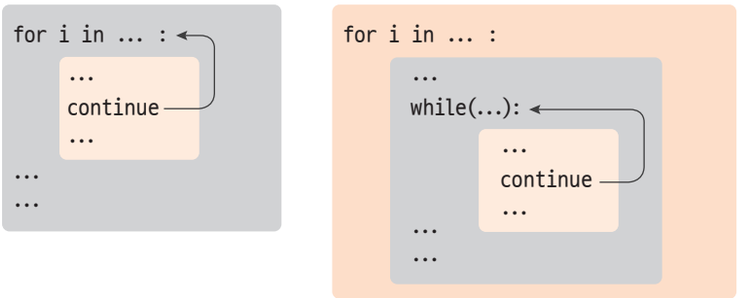
A-14.py

```
01 i = 1
02 sum = 0
03 while True :
04    sum += i
05    if sum > 3000:
06       break # 합이 3000을 넘어섰기 때문에 반복 종료
07    i += 1
08
09 print("1~%d까지 합하면 %d입니다." % (i, sum))
```

```
pi@pi:~/appendixA $ python A-14.py
1~77까지 합하면 3003입니다.
pi@pi:~/appendixA $
```

반복문 내의 continue 문

continue 문은 반복문 내에서 블록 코드를 실행하던 중 남은 블록 코드의 실행을 중단하고 다음 반복으로 넘어가고자 할 때 사용한다. 반복문이 중첩된 경우 continue 문을 직접 포함하는 한 개의 반복문에서 다음 반복으로 넘어간다. [그림 A-3]은 continue 문에 의해 다음 반복으로 넘어가는 경우를 보여준다.



[그림 A-3] continue 문으로 현재 블록 코드의 실행을 중단하고 다음 반복으로 넘어가는 경우

예제 A-15 continue 문

1에서 10 사이의 정수 중에서 3으로 나눈 나머지가 1인 정수들만 더한 합 구하라.

A-15.py

```
01 print("1~10사이 3으로 나눈 나머지가 1인 수만 더하기")
02 print("-----------------------------------")
03
04 sum = 0
05 for i in range(1, 11): # i가 1에서 10까지 반복
06    if (i % 3) != 1:
07       continue
08    print(i, end = " ")
09    sum += i
10 print("의 합은 %d" % sum)
```

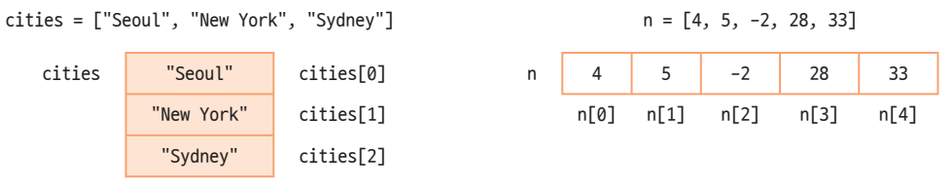
```
pi@pi:~/appendixA $ python A-15.py
1~10사이 3으로 나눈 나머지가 1인 수만 더하기
-----------------------------------
1 4 7 10 의 합은 22
pi@pi:~/appendixA $
```

A.6 컨테이너 객체 다루기

컨테이너 객체란 다수의 데이터를 저장하는 객체로 많이 활용되는 리스트(배열), 튜플, 딕셔너리에 대해 알아본다. 컨테이너의 가장 큰 특징은 저장되는 원소의 개수는 가변적이라는 점이다.

리스트

리스트(list)는 다수원소들을 연속적으로 저장하고 전체를 하나의 단위로 다루는 데이터 구조이다. 리스트는 다른 언어의 배열(array)과 유사하지만, 크기가 가변적이라는 점에서 다르다. [그림 A-4]는 2개의 리스트, cities와 n을 만드는 코드를 보여준다. cities는 3개의 문자열을 저장하는 리스트이고, n은 5개의 정수를 저장하는 리스트이다.



[그림 A-4] 리스트 사례

● 리스트 생성

[] 연산자를 사용하여 리스트를 쉽게 만들 수 있다. []에는 만들고자 하는 원소들의 초깃값들을 나열하면 된다.

```
members = [] # 빈 리스트 생성
cities = ["Seoul", "New York", "Syndey"]
week = ["월", "화", "수", "목", "금", "토", "일"]
plots = [-10, -20, 5, 10, 30]
```

파이썬 리스트에는 동일한 타입의 데이터만 저장할 수 있는 것이 아니라 다른 타입의 데이터도 다음과 같이 저장할 수 있다.

```
yoyo = [1, "황기태", 2, "이재문", 3, 4, 5] # 서로 다른 타입의 데이터 저장 가능
```

● 리스트의 각 원소 접근

리스트의 각 원소는 [] 기호와 0에서 시작하는 인덱스를 이용하여 접근한다. cities 리스트의 경우 원소의 개수가 3이므로 사용 가능한 인덱스는 0~2까지이다. cities 리스트의 개별 원소를 액세스하는 예는 다음과 같다.

```
name = cities[0]          # name은 "Seoul"
cities[1] = "Gaineville"  # "New York" 자리에 "Gaineville"이 저장됨
```

● 리스트의 원소 개수

리스트는 가변 크기이므로, 현재 저장된 원소의 개수를 알아내는 방법이 필요하다. len() 함수는 리스트의 원소 개수를 리턴한다. 다음은 len() 함수로 리스트의 원소 개수를 알아내는 예이다.

```
print( len( members ) )    # 0 출력
print( len( week ) )       # 7 출력
print( len( plots ) )      # 5 출력
```

● 리스트를 다루는 함수

파이썬은 리스트를 다루는 다양한 내장 함수를 제공하며 〈표 A-7〉은 주요 함수를 보여준다.

〈표 A-7〉 리스트를 다루는 다양한 함수들

함수	내용
list.append(element)	list의 맨 마지막에 element 삽입
list.pop()	list의 맨 마지막 원소 제거하고 그 값 리턴
list.insert(index, element)	list의 index 번째 원소로 element 삽입
list.remove(element)	list에서 첫 번째로 만나는 element 제거
del list[index]	list에서 index 번째 원소 제거
list.sort(reverse)	list의 원소들 정렬한다. reverse가 False이면 오름차순이고 True이면 내림차순이며 디폴트는 False
list.reverse()	list의 원소들을 역순으로 재정렬. list 내 원소 순서가 변경
len(list)	list에 저장된 원소들의 개수를 리턴

예제 A-16 리스트 다루기

리스트를 다루는 다양한 예이다.

A-16.py

```
01 list = [1, 2, "jmlee", 3] # 리스트 만들기
02 print(list)
03 list.append(4) # 마지막 원소로 4 추가
04 print(list)
05
06 list.insert(3, "d") # 인덱스 3위치에 "d" 추가
07 print(list)
08
```

```
09 list[list.index("jmlee")] = 10 # "jmee"를 10으로 수정
10 list.remove("d") # "d" 원소 삭제
11 print(list)
12
13 list.sort(reverse = True) # 내림차순으로 정렬
14 print(list)
15 print("원소의 개수 = %d" % len(list)) # 원소 개수
```

```
pi@pi:~/appendixA $ python A-16.py
[1, 2, 'jmlee', 3]
[1, 2, 'jmlee', 3, 4]
[1, 2, 'jmlee', 'd', 3, 4]
[1, 2, 10, 3, 4]
[10, 4, 3, 2, 1]
원소의 개수 = 5
pi@pi:~/appendixA $
```

튜플

튜플(tuple)은 리스트와 유사하다. 단지 다음 2가지 면이 다르다.

- 튜플은 ( ) 기호를 사용한다.
- 튜플은 원소의 값을 수정할 수 없다.

튜플을 생성하고 활용하는 코드는 다음과 같다.

```
cities = ("Seoul", "New York", "Syndey") # 3개의 원소로 구성되는 튜플
my_home_town = cities[0]                 # my_home_town = "Seoul"
```

하지만, 리스트와 달리 튜플은 처음 만든 상태에서 어떤 것도 수정할 수 없다. 즉 원소의 값을 수정할 수도, 원소를 삽입하거나 삭제할 수도 없기에, sort, insert, remove, pop과 같은 함수가 없다. 이런 이유로 튜플은 처음 만든 원소들의 값을 수정하지 않고 계속 사용할 필요가 있는 상황에 적합하다. 다음은 튜플을 사용하였을 때의 오류 상황이다.

오류
```
cities = ("Seoul", "New York", "Syndey")
del cities[0]             # 오류. 원소 삭제 불가
cities[1] = "Gaineville"  # 오류. 원소 값 수정 불가
```

하지만, 다음과 같이 새로운 튜플을 만드는 것은 가능하다.

```
cities = ("Seoul", "New York", "Syndey")
others = ("Busan", "Paris")
total = cities + others # total = ("Seoul", "New York", "Syndey", "Busan", "Paris")
```

딕셔너리

최근 들어 많이 사용되는 컨테이너가 딕셔너리(Dictionary)이다. 딕셔너리는 리스트나 튜플과 달리 〈key, value〉의 쌍을 하나의 원소로 저장하는 컨테이너이다. 리스트에서는 인덱스로 원소를 찾지만, 딕셔너리는 key 값으로 원소를 검색한다. 딕셔너리에는 key가 동일한 원소들이 중복하여 저장될 수 없다. 딕셔너리를 다루는 다양한 내장 함수는 〈표 A-8〉과 같다.

● 딕셔너리 생성

{} 기호를 사용하여 딕셔너리를 쉽게 만들 수 있다. {} 내에는 만들고자 하는 원소들의 초깃값(key: value)들을 나열하면 된다. members, week와 nation에 대한 딕셔너리를 만들어 보자.

```
members = {}   # 빈 딕셔너리
week = {"월":1, "화":2, "수":3, "목":4, "금":5, "토":6, "일":0}
nation = { "Korea":"Seoul", "Japan":"Tokyo", "France":"Paris"}
```

● 딕셔너리 원소 접근

딕셔너리에서 원소는 [] 기호를 이용하여 다음과 같이 쉽게 접근한다.

```
dict = {}          # 빈 딕셔너리 dict 생성
dict[key] = value  # 한 쌍의 새로운 <key, value> 원소를 딕셔너리에 저장
val = dict[key]    # 저장되어 있는 <key, value>에서 value를 읽어 val에 저장
```

여기서, 딕셔너리에서 dict[key]의 값을 읽어올 때 주의해야 한다. 이미 〈key, value〉가 저장되어 있다면 문제가 없으나, 저장되어 있지 않으면 key에 해당하는 value를 찾을 수 없기에 파이썬 프로그램은 런타임 오류를 발생하면서 종료된다. 이런 경우, 미리 dict.get(key)를 호출하여 key 값을 가진 원소가 딕셔너리에 저장되어 있는지 확인해야 한다.

〈표 A-8〉 딕셔너리를 다루는 다양한 함수들

함수	내용
dict.keys()	딕셔너리에 저장된 모든 키들을 저장한 dict_keys 객체 리턴
dict.values()	딕셔너리에 저장된 모든 value들을 저장한 dict_values 객체 리턴
dict.items()	딕셔너리에 저장된 (키, 값)에 대한 dict_items 객체 리턴
dict.clear()	딕셔너리의 모든 원소 삭제
dict.get(key)	key로 저장된 원소가 있으면 value 리턴, 없으면 None리턴
del dict[key]	dict[key]를 제거

예제 A-17 딕셔너리 다루기

딕셔너리에서 최고 점수 또는 최저 점수를 찾는 프로그램이다.

A-17.py

```
01 dict = {"jmlee": 88, "kitae": 98, "ihjung":99}
02
03 cmd = input("high 또는 low를 입력하세요>> ")
04
05 names = list(dict.keys())                # dict의 모든 키들을 1개의 리스트로 만들기
06 scores = list(dict.values())             # dict의 모든 값들을 1개의 리스트로 만들기
07 if cmd == "high":
08       high = 0
09       for i in range(len(names)):
10             high = scores[i] if high < scores[i] else high
11       print("최고 점수는 %d입니다." % high)
12
13 elif cmd == "low":
14       low = 100
15       for i in range(len(names)):
16             low = scores[i] if low > scores[i] else low
17       print("최저 점수는 %d입니다." % low)
18
19 value = dict.get("gdhong")               # "gdhong"이 dict에 없을 때 value는 None이 됨
20 print(value)
```

```
pi@pi:~/appendixA $ python A-17.py
high 또는 low를 입력하세요>> low
최저 점수는 88입니다
None
pi@pi:~/appendixA $
```

컨테이너와 for 문

컨테이너에 저장된 원소들에 대해 반복 연산을 하고자 하는 경우, 다음과 같은 형식으로 for 문을 활용할 수 있다.

```
for 변수 in 컨테이너:
    반복될 문장1
    ...
    반복될 문장n
```

컨테이너에 해당하는 리스트, 튜플, 딕셔너리 모두 작동한다. 리스트나 튜플의 경우, for 반복이 진행되는 동안 '변수'에는 리스트나 튜플의 원소들이 순서대로 하나씩 저장되어 반복되며, 딕셔너리의 경우, '변수'에는 key들이 하나씩 저장되어 반복된다.

예제 A-18 컨테이너 응용

딕셔너리와 배열에서 최고 점수 또는 최저 점수를 찾는 프로그램이다.

A-18.py

```
01 students = {"jmlee": [88, 97, 50], "kitae": [90, 80], "ihjung": [99, 70, 60]}
02
03 cmd = input("high 또는 low를 입력하세요>> ")
04
05 if cmd == "high":
06       high = 0
07       for name in students.keys():
08             scores = students[name]
09             for score in scores:
10                   high = score if high < score else high
11       print("최고 점수는 %d입니다." % high)
12
13 else:
14       low = 100
15       for name in students.keys():
16             scores = students[name]
17             for score in scores:
18                   low = score if low > score else low
19       print("최저 점수는 %d입니다." % low)
```

```
pi@pi:~/appendixA $ python A-18.py
high 또는 low를 입력하세요>> high
최고 점수는 99입니다.
pi@pi:~/appendixA $
```

A.7 함수

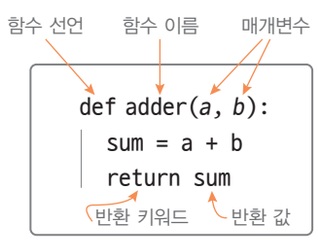
함수(function)는 데이터를 전달받아 처리한 후 결과를 돌려주는 코드 블록이다. 예를 들어 더하기의 천재인 '애더'에게 친구들은 가끔 덧셈을 부탁한다. 친구들은 '애더'에게 왼손과 오른손에 들린 종이에 수를 적어 보여주고 계산을 시킨다. 그리고 계산 결과를 받는다.

'애더'가 전달받은 두 수를 더하고 결과를 말해주는 것처럼, 파이썬에서 데이터를 전달받아 정해진 작업을 수행하고 그 결과를 돌려주도록 작성된 코드 블록이 바로 함수이다. 함수는 전달받는 데이터가 없을 수도 있고, 돌려주는 결과가 없을 수도 있다.

함수의 구성

파이썬 함수는 [그림 A-5]와 같은 형식으로 작성한다.

```
def 함수이름(arg1, agr2, ..., argn):
    ... 프로그램 코드 ...
    결과를 돌려주는 return 문
```



[그림 A-5] 함수의 구성과 함수 작성 사례

[그림 A-5]의 함수의 구성 요소에 대해 자세히 알아보면 다음과 같다.

- def - 함수의 선언을 표시하는 키워드. def 문의 마지막에 콜론(:)을 사용하여 함수의 시작임을 알려야 한다.
- 함수 이름 - 개발자가 정하는 것으로 함수의 목적에 맞게 이름을 정하면 된다.
- arg1, agr2, ..., argn - 함수를 호출하는 곳으로부터 값을 전달받는 매개변수이다. 여러 개가 있는 경우 콤마(,)로 분리하면 된다.

- 프로그램 코드 - 함수의 작업을 실행하는 파이썬 코드이다. 이것은 들여쓰기를 준수하여 블록임을 표시하여야 한다.
- return 문 - 함수의 실행을 종료하고 호출한 곳으로 되돌아가면서 함수의 실행 결과를 반환하는 문이다.

함수 호출

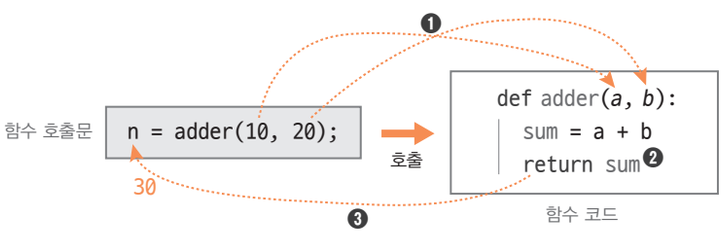
함수에게 일을 지시하고 결과를 받는 것을 함수 호출(function call)이라 한다. '애더'가 요청받지 않으면 덧셈을 하지 않는 것처럼, 함수 역시 호출하지 않으면 실행되지 않는다. 함수의 호출에는 다음 2가지 방법이 있다.

- 변수이름 = 함수이름(arg1, arg2, ..., argn) # 함수 호출 후 함수로부터 리턴 값을 받는다.
- 함수이름(arg1, arg2, ..., argn) # 함수 호출 후 함수로부터 리턴 값을 받지 않는다.

함수가 아무 값도 리턴하지 않거나, 리턴한다고 해도 리턴받는 값이 필요 없는 경우 두 번째 경우처럼 호출하면 된다. adder() 함수를 호출하는 다음 코드를 통해 실행 과정을 알아보자.

```
n = adder(10, 20) # adder() 함수 호출
```

[그림 A-6]은 앞의 함수 호출이 실행되는 과정을 보여준다. adder(10, 20)을 호출하면, 10이 매개변수 a에, 20이 매개변수 b에 전달된다. 그리고 adder() 함수가 실행되어 합한 결과 30을 sum에 저장하고 이 값을 리턴한다. 리턴된 값은 변수 n에 저장된다.



[그림 A-6] adder() 함수의 호출과 adder() 함수의 실행 과정

매개변수의 디폴트 값

함수를 호출할 때 매개변수에 값을 넘기는 것을 생략하면, 미리 정해진 값이 넘어오도록 함수를 선언할 때 매개변수에 디폴트 값을 설정할 수 있다. 앞의 adder() 함수의 두 번째 매개변수 b에 디폴트 값 20을 설정해보자.

```
def adder(a, b=20):
   sum = a + b
   return sum
```

그러면, 다음과 같이 adder() 함수를 호출할 수 있다.

```
sum = adder(10)        # adder(10, 20)과 동일, 매개변수 b에 디폴트 값 20 전달
sum = adder(10, 30)    # adder(10, 30)과 동일
```

add(10)은 다음과 같이 처리한다. 10을 먼저 매개변수에 대한 전달 값으로 해석하고, 두 번째 매개변수에 전달하는 값이 생략된 것을 인식하여 두 번째 매개변수에 지정된 디폴트 값 20을 전달한다.

매개변수에 디폴트 값을 적용하는 조건이 하나 있는데, 디폴트 값이 적용된 매개변수 뒤에는 디폴트 값이 없는 매개변수를 정의할 수 없다.

명명된 매개변수(named parameter)

파이썬은 함수 호출 시 매개변수에 값을 전달하는 2가지 방법을 제공한다.

- 위치 매개변수
- 명명된 매개변수(named parameter)

● 위치 매개변수

위치 매개변수(positional parameter)는 함수 호출문에 적시된 값들이 함수에 선언된 매개변수에 1:1로 순서대로 전달되는 것으로 이제까지 사용한 매개변수 전달방식이다. 앞서 만든 adder() 함수를 위치 매개변수 방식으로 호출하면 다음과 같다.

```
n = adder(20, 30)
```

● 명명된 매개변수

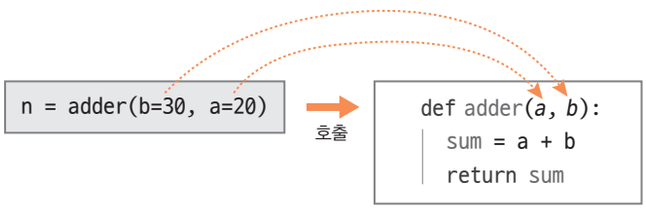
명명된 매개변수(named parameter)는 함수 호출문에 '매개변수 이름=값'을 이용하여 매개변수에 값을 전달하는 방식이다.

앞서 만든 adder() 함수를 명명된 매개변수 방식으로 호출하면 다음과 같다.

```
n = adder(a=20, b=30)
```

명명된 매개변수 방식을 사용하면 매개변수를 순서대로 전달할 필요가 없으므로 다음과 같이 호출해도 된다. [그림 A-7]은 이에 대한 매개변수의 전달 과정이다.

```
n = adder(b=30, a=20)
```



[그림 A-7] 명명된 매개변수로 매개변수 값 전달

명명된 매개변수 방식의 가장 큰 장점은, 함수에 선언된 매개변수의 순서를 기억하지 않아도 된다는 점이다. 개발자들은 가끔 매개변수의 순서를 착각하여 매개변수에 잘못된 값을 전달할 수 있는데, 명명된 매개변수 방식은 이런 착각으로 인한 오류의 가능성을 줄여준다. 그래서 최근 들어 여러 컴퓨터 언어에서 명명된 매개변수(Named parameter) 방식을 지원하고 있다.

한편, 파이썬에서는 위치 매개변수와 명명된 매개변수를 혼용하여 사용하는 것은 금지하고 있다.

오류
```
n = adder(20, b=100)      # 오류, 2가지를 혼용할 수 없다.
```

잠깐! 명명된 매개변수의 단점

명명된 매개변수를 사용할 시 매개변수의 정확한 이름을 알아야 한다. 예를 들어 area(width, height)라는 함수가 있을 때 area(w=100, h=200)으로 하면 오류가 된다. 이러한 단점을 해결하기 위하여 최근 많은 코드 편집기에서는 명명된 매개변수의 이름을 에디터가 보여준다.

예제 A-19 함수 만들기

구구단을 출력하는 함수 gugudan()을 작성하고, 사용자로부터 정수를 입력받아 이 함수를 호출하여 구구단을 출력하는 프로그램을 작성하라.

A-19.py

```
01 def gugudan(n):
02     for i in range(1,10):
03         print("%d x %d = %d" % (n, i, n*i))
04
05 a = input("구구단 몇단을 원하세요>> ")
06 n = int(a)
07 gugudan(n)
```

```
pi@pi:~/appendixA $ python A-19.py
구구단 몇단을 원하세요>> 6
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
pi@pi:~/appendixA $
```

전역 변수와 지역 변수

변수는 사용 범위에 따라 다음 2가지 종류가 있다.

- 전역 변수 - 함수 밖에 선언되는 변수, 프로그램이 실행을 시작할 때 생겨나서 프로그램 종료까지 생존하며, 모든 함수에서 사용하는 것이 가능하다.
- 지역 변수 - 함수 내에서 선언되는 변수, 함수가 시작되면 생겨나서 함수가 종료되면 사라지므로 선언된 함수 내에서만 사용하는 것이 가능하다.

● 전역 변수와 지역 변수 선언

함수 내에서 새로운 변수를 선언하면 지역 변수가 된다. 다음은 매우 기본적인 전역 변수와 지역 변수의 선언 및 사용 사례이다.

```
01 x = 10  # 전역 변수
02
03 def f():
04     y = 50          # 지역 변수
05     return y + 10   # 60 리턴
06
07 t = f()                              # 함수 f() 호출
08 print("전역 변수 x = ", x)           # 10 출력
09 print("함수 f()의 리턴 값은 ", t)    # 60 출력
```

```
전역 변수 x = 10
함수 f()의 리턴 값은 60
```

함수 f()의 바깥에 전역 변수 x가 선언되어 있고, 함수 f() 내에는 지역 변수 y가 선언되어 사용된다. 함수 f()는 60을 리턴하므로 print(t)는 60을 출력한다. 변수 y는 함수 바깥에서 사용할 수 없다.

● 함수 내에서 전역 변수 읽기

함수 내에서 당연히 전역 변수를 사용할 수 있다. 하지만, 함수 내에서 전역 변수의 값을 읽기만 경우와 전역 변수에 값을 쓰는 경우, 함수를 작성하는 개발자는 다르게 처리해야 한다.

함수 내에서 전역 변수의 값을 읽기만 하는 프로그램에 대해 알아보자.

```
01 x = 10  # 전역 변수이다.
02 def f():
03     y = 50          # 지역 변수
04     return x + y    # x는 전역 변수
05
06 t = f()                              # 함수 f() 호출
07 print("전역 변수 x = ", x)           # 10 출력
08 print("함수 f()의 리턴 값은 ", t)    # 60 출력
```

```
전역 변수 x = 10
함수 f()의 리턴 값은 60
```

함수 내에서 전역 변수를 읽기만 하는 경우 전역 변수 이름을 그냥 사용하면 된다.

● 함수 내에서 전역 변수에 쓰기

하지만, 만일 전역 변수에 값을 쓰는 경우, 반드시 global 키워드를 사용하여 전역 변수임을 명확히 하여야 한다. global 키워드를 사용하는 다음 예를 보자.

```
01 x = 10  # 전역 변수이다.
02
03 def f():
04     global x        # x는 전역 변수임을 알림
05     x = x + 10      # 전역 변수 x는 20
06     y = 50          # 지역 변수
07     return x + y    # 70 리턴
08
09 t = f()                              # 함수 f() 호출
10 print("전역 변수 x = ", x)           # 20 출력
11 print("함수 f()의 리턴 값은 ", t)    # 70 출력
```

```
전역 변수 x = 20
함수 f()의 리턴 값은 70
```

● 전역 변수 사용시 주의

만일 다음과 같이 global을 사용하지 않으면 오류가 발생한다.

```
x = 10 # 전역 변수이다

def f():
#   global x     # 주석 처리함, x가 전역 변수임을 알리지 않음
    x = x + 10   # x는 지역 변수인데 미 초기화로 오류가 발생
    y = 50       # 지역 변수
    return x + y # 80 리턴

t = f()                          # 함수 f() 호출
print("전역 변수 x = ", x)       # 10 출력
print("함수 f()의 리턴 값은 ", t) # 80 출력
```

오류

오류 메시지는 다음과 같다.

```
Traceback (most recent call last):
  File "/home/pi/appendixA/a.py", line 9, in <module>
    t = f()                      # 함수 f() 호출
  File "/home/pi/appendixA/a.py", line 5, in f
    x = x + 10   # x는 지역 변수인데 미 초기화로 오류가 발생
UnboundLocalError: local variable 'x' referenced before assignment
```

'UnboundLocalError'의 의미는 파이썬 인터프리터가 x = x + 10의 문장에서 x를 지역 변수로 인식하고, x에 어떤 값도 주어지지 않은 상태에서 x의 값을 읽어 10을 더하는 연산을 수행할 수 없다는 뜻이다.

함수 내에서 전역 변수를 사용할 때 주의하여야 한다. 특히 전역 변수에 쓰기를 하는 경우 반드시 global 키워드로 전역 변수임을 알려야 한다. 그러므로 저자는, 함수 내에서 전역 변수를 사용하는 경우, 읽기만 하던 쓰기를 하던 무조건 global 키워드를 사용하여 전역 변수임을 명확히 하기를 추천한다. 그러면 코드의 가독성이 높아지고 후에 변수 이름으로 인한 실수도 줄어든다.

A.8 모듈

모듈이란?

파이썬에서 모듈(module)이란 하나의 파이썬 프로그램 파일(.py 파일)을 뜻한다. 그러므로 하나의 모듈을 구성하는 요소들은 파이썬 프로그램 파일 내에 만들어진 전역 변수, 함수, 클래스들이다. 파이썬 프로그램 파일 이름에서 확장자 .py를 뺀 이름을 '모듈 이름'으로 한다. 그리고 모듈의 전역 변수를 특별히 모듈의 프로퍼티(property)라고 부른다.

import 키워드로 모듈 임포트

모듈은 그 자체로도 실행할 수 있지만, 하나의 모듈에서 다른 모듈을 사용하기 위해서는, 사용하고자 하는 모듈을 임포트하여야 한다. 모듈을 임포트하기 위해서는 import 키워드를 이용한다.

다음과 같이 a.py 프로그램은 단독으로 실행될 수 있고, import 되어 모듈로 사용될 수 있다.

```
# 모듈 a. a.py 파일에 저장
name = "황기태" # 모듈 a의 전역 변수

def printName():
   print("이름 = %s" % name)

printName()
```

● 단독으로 실행되는 경우

모듈 a를 실행하면 이제까지 파이썬 프로그램과 같이 실행된다.

```
pi@pi:~/appendixA $ python a.py
이름 = 황기태
pi@pi:~/appendixA $
```

● 모듈로 임포트되어 실행되는 경우

다음 코드와 같이 프로그램 b.py(모듈 b)에서 모듈 a를 임포트하는 경우, 프로그램 b.py(모듈 b)는 모듈 a를 임포트 한 후, 모듈 a를 1번 실행한다.

```
# 모듈 b. b.py 파일에 저장
import a                    # 임포트할 때 모듈 이름만 사용

name = "이재문"             # 모듈 b의 전역 변수

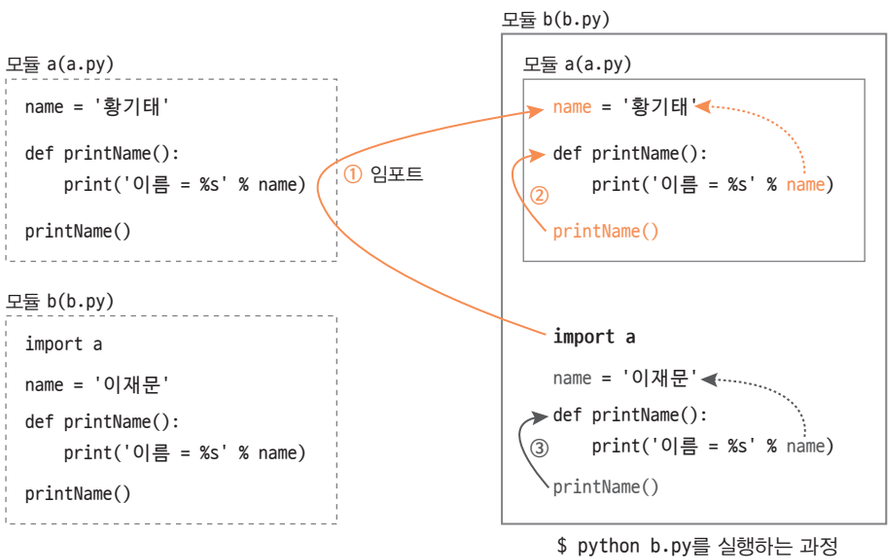
def printName():
   print("이름 = %s" % name)

printName()
```

따라서 모듈 b를 실행하면 다음과 같이 출력된다.

```
pi@pi:~/appendixA $ python b.py
이름 = 황기태
이름 = 이재문
pi@pi:~/appendixA $
```

이 과정은 [그림 A-8]에 잘 나타나 있다. 모듈 a가 임포트될 때(그림에서 ①), 한번 실행되므로 a.py에 있는 name이라는 변수에 '황기태'가 저장되고, printName()가 호출되었고(그림에서 ②), 그에 따라 이 함수는 name 변수의 값 "황기태"를 출력하였다. 그런 후, b.py가 실행되므로 b.py에 있는 name이라는 변수에 '이재문'이 저장되고, printName()이 실행되어(그림에서 ③), 모듈 b에 있는 name의 값 "이재문"을 출력하였다.



[그림 A-8] 프로그램 b.y(모듈 b)에서 모듈 a를 import 할 때 전체 실행 과정

● 임포트된 모듈 활용

모듈을 임포트하면, 임포트된 모듈의 전역 변수와 함수, 클래스들을 활용할 수 있다. 이때 임포트된 모듈의 전역 변수나 함수 등을 액세스할 때, 다음과 같이 점(.)을 이용한다.

```
모듈이름.전역변수
모듈이름.함수
```

점(.)은 전역 변수나 함수가 어떤 모듈에 속한 것인지를 구분한다. 다음은 모듈 a를 임포트하는 다른 모듈 c를 작성한 사례이다.

```
# 모듈 c. c.py에 저장
import a

name = "이재문"

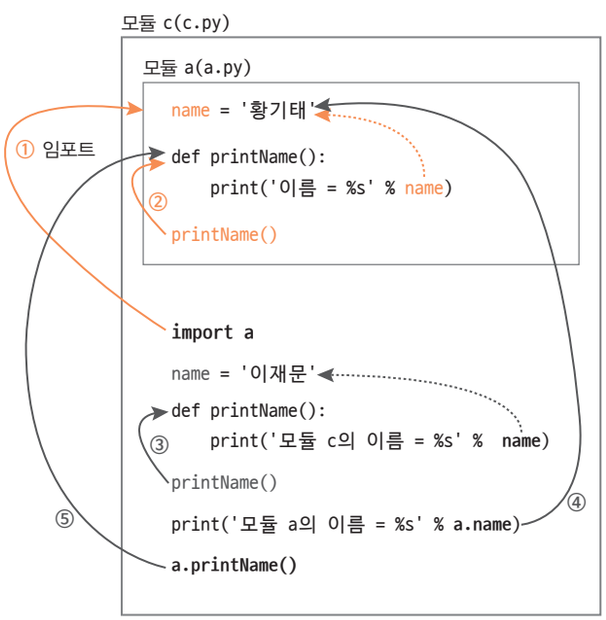
def printName():
   print("모듈 c의 이름 = %s" % name)    # 모듈 c의 name, "이재문" 출력

printName()                              # 모듈 c의 printName() 함수 실행

print("모듈 a의 이름 = %s" % a.name)     # 모듈 a의 name, "황기태" 출력
a.printName()                            # 모듈 a의 printName() 함수 실행
```

모듈 c를 실행하면 다음과 같으며, [그림 A-9]와 같이 모듈의 요소들이 활용된다.

```
pi@pi:~/appendixA $ python c.py
이름 = 황기태
모듈 c의 이름 = 이재문
모듈 a의 이름 = 황기태
이름 = 황기태
pi@pi:~/appendixA $
```



$ python c.py를 실행하는 과정

[그림 A-9] 모듈 c가 실행되는 전체 과정

'from 모듈이름 import *' 등으로 모듈 임포트

모듈을 임포트하는 또 다른 방식은 다음과 같다.

- from 모듈이름 import * 또는 모듈에 포함된 전역 변수, 함수의 콤마 목록

[그림 A-9]에서 볼 수 있듯이 'import 모듈이름'으로 임포트하는 경우 임포트된 모듈 내의 모든 전역 변수나 함수를 액세스할 수 있다.

그러나 'from 모듈이름 import ...'에서는 다음과 같은 제약사항이 있다.

- 제약사항1 : 임포트 될 모듈(a)과 임포트하는 모듈(b 또는 c)에서 동일한 식별자(전역 변수 이름, 함수이름)가 있는 경우 임포트될 모듈(a)의 식별자는 무시된다.
- 제약사항2 : 임포트될 모듈에서 _로 시작하는 식별자는 임포트하는 모듈에서 액세스할 수 없다.

이러한 제약사항도 있지만, 모듈에 포함된 다양한 전역 변수나 함수 중에서 필요한 부분만 임포트할 수 있다. 이렇게 임포트된 전역 변수나 함수는 모듈 이름 표시 없이 바로 사용할 수도 있다.

다음은 모듈 a를 from을 사용하여 임포트하는 모듈 d를 작성한 사례이다.

```
# 모듈 d. d.py 파일에 저장

from a import name, printName

def printName():
   print("성함 = %s" % name) # name은 a모듈의 name

printName()    # printName() 함수는 이 모듈의 printName()임
```

모듈 a에서 name과 printName() 함수만을 임포트 한 경우이다. 모듈 d에서 별도로 name이라는 변수를 선언하지 않았으므로 모듈 d에서의 name은 모듈 a에서 선언한 name이다. 반면 printName() 함수는 모듈 a, d 모두에 선언되어 있다. 즉, 중복이 발생한 경우이다. 이 경우 임포트된 모듈 a의 printName() 함수는 무시된다. 따라서 다음과 같이 출력됨을 알 수 있다.

```
pi@pi:~/appendixA $ python d.py
이름 = 황기태
성함 = 황기태
pi@pi:~/appendixA $
```

첫 라인의 출력은 모듈 a가 임포트 되면서 모듈 a의 printName() 함수의 출력이고, 두 번째 라인의 출력은 모듈 d의 마지막 라인에서 printName() 함수의 호출에 의한 출력이다. 이때 printName() 함수의

호출은 모듈 d의 printName() 함수를 실행시키므로 '이름' 대신에 '성함'이 출력됨을 알 수 있다.

__name__ 전역 변수 활용

모든 파이썬 모듈에는 자동으로 선언되는 __name__이라는 전역 변수가 있다. 프로그램의 실행 초기에 이 변수에 저장되는 값이 자동으로 결정되는데, 다음과 같이 2가지 상황에 따라 서로 다른 값이 저장된다.

- 모듈이 단독으로 실행될 때, __name__ 변수에는 "__main__" 문자열 저장
- 모듈이 임포트되어 실행될 때, __name__ 변수에는 "모듈이름"의 문자열 저장

다음과 같이 a.py를 수정해보자.

```
# 모듈 a. a.py 파일에 저장
name = "황기태" # 모듈 a의 전역 변수

def printName():
   print("이름 = %s" % name)

if __name__ == "__main__":  # 모듈 a가 단독으로 실행되면 조건문은 True
   printName()
```

a.py를 단독으로 실행한다면 printName()가 호출되어 다음과 같이 실행된다.

```
pi@pi:~/appendixA $ python a.py
이름 = 황기태
pi@pi:~/appendixA $
```

하지만, a.py가 b.py에 의해 임포트되는 경우,

```
# 모듈 b. b.py 파일에 저장
import a # 임포트할 때 모듈 이름만 사용

name = "이재문" # 모듈 b의 전역 변수

def printName():
   print("이름 = %s" % name)

printName()
```

모듈 a의 __name__의 값이 'a'가 되므로, 조건문(if __name__ == "__main__")이 False가 되어, 모듈 a의 printName()이 호출되지 않았다.

```
pi@pi:~/appendixA $ python b.py
이름 = 이재문
pi@pi:~/appendixA $
```

잠깐! **__name__ 전역 변수**

__name__ 변수는 모듈이 잘 작동하는지, 모듈 테스트용으로 많이 활용된다.

예제 A-20 임포트 사례

다음과 같이 작성된 mymodule.py를 실행하였을 때 예상되는 출력과 mymodule을 임포트한 A-20.py를 실행할 때 예상되는 출력은 각각 무엇인가?

mymodule.py

```
01 delta = 10 # 전역 변수
02
03 def plus(value):
04     global delta # delta는 전역 변수임을 알림
05     delta += value
06     return delta
07
08 if __name__ == "__main__" :
09     print("mymodule 테스트 %d" % plus(100))
10
11 if __name__ == "mymodule" :
12     print("mymodule imported...\n delta is initialized to 10\n")
```

A-20.py

```
01 import mymodule
02
03 ret = mymodule.plus(10)
04 print(ret)
05
06 ret = mymodule.plus(10)
07 print(ret)
```

```
pi@pi:~/appendixA $ python mymodule.py
mymodule 테스트 110
pi@pi:~/appendixA $
```

```
pi@pi:~/appendixA $ python A-20.py
mymodule imported...
 delta is initialized to 10

20
30
pi@pi:~/appendixA $
```

A.9 파일 입출력

파이썬 언어는 새 파일을 만들거나, 파일에서 읽거나 쓰는 등의 기능을 제공한다. 파일은 바이너리 파일과 텍스트 파일로 나뉘고 이들을 읽고 쓰는 방법은 조금씩 다른데, 이 절에서는 텍스트 파일에 대해서만 다룬다. 텍스트 파일은 텍스트 데이터만 저장하는 파일이다. 파이썬은 파일 입출력을 위해 〈표 A-9〉와 같이 open(), read(), write(), close(), readline(), readlines(), writelines() 등의 함수를 제공한다.

〈표 A-9〉 파이썬에 내장된 파일 처리 함수

내장함수	내용
file = open(name, mode)	name 파일을 mode의 값에 따라 생성 혹은 열기. file 객체를 리턴하며, 추후 입출력은 file 객체 활용
str = file.read()	file 객체가 가리키는 파일의 모든 데이터를 읽어 하나의 문자열(str)로 리턴
file.write(str)	str의 문자열을 file 객체가 가리키는 파일에 저장
lines = file.readlines()	file 객체가 가리키는 파일의 모든 데이터를 라인별로 읽어 문자열 배열(lines)로 리턴, 각 문자열(라인) 끝에는 '\n' 포함
file.writelines([s1,s2,.., sn])	file 객체가 가리키는 파일에 s1, s2, sn의 문자열을 순차적으로 저장. s1, s2, sn의 끝에는 '\n'이 포함되어 있어야 함
file.close()	file 객체가 가리키는 파일을 입출력을 종료하기 위해 닫음

파일 열기와 읽기/쓰기

파일을 만들거나 읽거나 쓰기 위해서는 먼저 파일을 여는 작업이 가장 먼저 진행되어야 한다. 파일 열기(file open)란 파일의 데이터를 읽거나 쓰기 위해 시스템에서 필요한 준비 작업을 진행하는 과정이다.

일반적으로 파일에 쓰는 작업의 전형적인 파이썬 프로그램의 모양은 다음과 같다.

```
file = open(fname, "w")
for i in range(...):
   file.write("...")
file.close()
```

먼저 open() 함수에 쓰고자 하는 파일의 이름 fname과 쓰기임을 나타내는 "w" 모드를 지정하여 파일을 열고, 필요한 만큼 루프를 돌면서 write() 함수를 활용하여 파일에 쓴다. open()의 2번째 매개변수는 파일을 열 때 지정하는 열기 모드로 〈표 A-10〉과 같이 다양하다. 열기 모드는 파일을 왜 혹은 어떤 식으로 입출력하고자 하는지 그 이유를 지정한다. 파일 쓰기가 끝나면 close() 함수를 호출하여 파일을 닫는다. fname의 파일이 기존에 없었다면 새로 만들고 처음부터 쓴다. 만일 fname의 파일이 있었다면 이 파일이 처음부터 쓰기 때문에 기존의 파일 데이터를 사라진다.

파일을 읽는 전형적인 파이썬 프로그램의 모양은 다음과 같다.

```
file = open(fname, "r")
for line in file.readlines(): 파일 전체를 읽고, 각 라인별로 line에 저장
   line에 저장된 문자열을 처리하는 코드들 ...
file.close()
```

〈표 A-10〉 open 함수의 2번째 매개변수 mode의 값

열기 모드	내용
"r"	'읽기 모드'라고 부르며, 파일을 읽을 때 사용. mode 값이 생략되면 디폴트가 "r"이다.
"w"	'쓰기 모드'. 파일에 데이터를 쓸 때 사용. 이미 파일이 존재하면 쓰기 시작점이 맨 앞으로 설정되고 뒤에 데이터는 삭제됨. 파일이 존재하지 않는다면 새 파일을 만들고 쓰기 시작점을 맨 앞으로 설정
"a"	'추가 쓰기 모드'. "w"와 달리 기존에 존재하는 파일이면 파일의 끝을 쓰기 시작점으로 설정하여 파일 끝에서부터 이어 쓰기 진행
"x"	파일이 없으면 파일을 생성하고 '쓰기 모드'로 열기. 파일이 있으면 오류 발생
"b"	오디오나 비디오와 같은 바이너리 파일의 입출력을 위한 '바이너리 모드' 설정
"t"	텍스트 파일의 입출력을 위한 '텍스트 모드' 설정. "b"나 "t"가 주어지지 않으면 디폴트가 "t"임

예제 A-21 간단한 파일 읽기/쓰기 연습

myfile.txt 파일을 새로 생성하고 여러 줄의 문자열을 저장하라. 그리고 myfile.txt 파일을 다시 읽어 출력하는 프로그램을 작성하라. 그리고 cat 명령으로 myfile.txt 파일을 화면에 출력하여 제대로 저장되었는지 확인하라.

A-21.py

```
01 names = ["손흥민", "황희찬", "박항서", "유상철", "김남일"]
02 file = open("myfile.txt", "w") # 쓰기 모드로 myfile.txt 파일 열기. myfile.txt가 없으면 생성
03
04 for i in range(0, len(names)): # len(names)가 5이므로 5번 반복
05    line = "%d번째 이름 %s\n" % (i, names[i])
06    file.write(line) # 파일 쓰기
07
08 file.close() # 파일 닫기. 다시 열기 전에는 쓸 수 없음
09
10 # 파일 열기
11 file = open("myfile.txt", "r") # 읽기 모드로 myfile.txt 파일 열기. myfile.txt 파일이 없으면 예외 발생
12
13 for line in file.readlines(): # 파일에서 각 라인을 반복하여 line 변수에 읽음
14    print(line, end="") # 읽은 라인을 화면에 출력
15
16 file.close()  # 파일 닫기. 다시 열기 전에는 읽을 수 없음
```

```
pi@pi:~/appendixA $ python A-21.py
0번째 이름 손흥민
1번째 이름 황희찬
2번째 이름 박항서
3번째 이름 유상철
4번째 이름 김남일
pi@pi:~/appendixA $
```

```
pi@pi:~/appendixA $ cat myfile.txt
0번째 이름 손흥민
1번째 이름 황희찬
2번째 이름 박항서
3번째 이름 유상철
4번째 이름 김남일
pi@pi:~/appendixA $
```

이론문제

1. 다음 설명 중 틀린 것은?

① 파이썬은 리눅스 운영체제에서만 사용할 수 있다.

② 파이썬은 인터프리터 방식으로 실행된다.

③ 파이썬 프로그램은 컴파일될 필요가 없다.

④ 파이썬 프로그램은 두 가지 방법으로 실행할 수 있다.

2. 다음 설명 중 틀린 것은?

① a = {}에서 a는 빈 딕셔너리이다.

② a = ["jmlee": 80, "kitae": 90]은 딕셔너리 a를 초기화하는 것이다.

③ a["jmlee"]는 딕셔너리 a에서 키가 "jmlee"인 요소의 값을 말한다.

④ x = a["ihjung"]를 할 때 a["ihjung"]의 값이 존재하지 않으면 x에 None이 저장된다.

3. 파이썬 데이터형 변환에 대하여 틀린 것은?

① int("10")은 문자열 "10"을 정수 10으로 변환한다.

② float("10")은 문자열 "10"을 실수 10.0으로 변환한다.

③ int("10.0")은 문자열 "10.0"을 정수 10으로 변환한다.

④ float("10.0")은 문자열 "10.0"을 실수 10.0으로 변환한다.

4. 다음 중 가장 틀린 것은?

① str(value)는 value를 문자열로 변환한다.

② chr(value)는 value를 문자로 변환한다.

③ ord(value)는 value를 아스키코드로 변환한다.

④ bool(value)는 value가 0이 아니면 True로 변환한다.

5. 연산자에 대한 설명 중 틀린 것은?

① a % b는 a를 b로 나눌 때 나머지이다.

② a // b는 a를 b로 나눌 때 몫이다.

③ a / b는 a를 b로 나눌 때 몫이다.

④ 11 // 4.5의 결과는 2.0이다.

연습문제

6. 다음 연산 결과에 대하여 틀린 것은?

① "raspberry"+"pi"의 결과는 "raspberrypi"이다.

② "raspberry"+3.14의 결과는 "raspberry3.14"이다.

③ str(1988)+"응답하라"의 결과는 "1988응답하라"이다.

④ str(19)+str(88)의 결과는 "1988"이다.

7. 다음 코드에서 틀린 부분을 모두 고쳐라.

```
if year == 4
      print("Senior")
else if year == 3
      print("Junior")
else if year == 2
      print("Sophomore")
else
      print("Freshman")
```

8. 다음은 리스트의 함수에 대한 설명이다. 틀린 것은?

① a.push(10)는 리스트 a의 마지막에 10을 저장한다.

② a.pop()는 리스트 a의 마지막 원소를 제거한 후 이 원소를 리턴한다.

③ a.remove(10)은 리스트 a의 원소 중 10인 것을 제거한다.

④ a.sort(reverse=True)는 리스트 a의 원소들을 내림차순으로 정렬한다.

9. 다음 프로그램은 실행 오류를 일으킨다. 오류가 발생하지 않도록 수정하라.

```
dict = {"김동준": 100, "박경민": 90, "박민서": 89}
if dict["황호연"] > 80 :
   print("황호연은 우수한 학생입니다.")
else:
   print("황호연은 더 노력하여야 합니다.")
```

10. myfunc 함수의 코드가 다음과 같을 때, myfunc 함수의 호출에 대하여 설명한 것 중 틀린 것은?

```
def myfunc(a, b = 20):
   reurn a - b
```

① myfunc(10)은 -10을 리턴한다.
② myfunc(b=30, a=20)은 10을 리턴한다.
③ myfunc(b=30)은 오류이다.
④ myfunc(a=10)은 -10을 리턴한다.

실습문제

1. 다음은 이름 문자열을 저장하고 있는 names 리스트이다.

```
names = ["SonHeungmin", "HwangHeechan", "ParkSung", "HongSuhon", "HongMyungbo"]
```

names에서 가장 긴 이름을 찾아 출력하는 프로그램을 작성하라. 실행 예는 다음과 같다.

```
pi@pi:~/prac/appendixA $ python pracA-1.py
예제 문자열: SonHeungmin, HwangHeechan, ParkSung, HongSuhon, HongMyungbo
가장 긴 문자열: HwangHeechan
pi@pi:~/appendixA $
```

2. 다음은 정수가 저장된 리스트이다.

```
list = [18, 19, 5, 18, 18, 13, 15, 2, 11, 19]
```

이 리스트에서 값들의 개수를 출력하는 프로그램을 작성하라. 실행 예는 다음과 같다.

```
pi@pi:~/prac/appendixA $ python pracA-2.py
list: 18 19 5 18 18 13 15 2 11 19
<18: 3개> <19: 2개> <5: 1개> <13: 1개> <15: 1개> <2: 1개> <11: 1개>
pi@pi:~/appendixA $
```

연습문제

3. 딕셔너리를 이용하여 다음 실행 사례와 같이 작동하도록 이름과 전화번호를 관리하는 프로그램을 작성하라.

```
pi@pi:~/prac/appendixA $ python pracA-3.py
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 1
이름을 입력하세요: jmlee
전화번호를 입력하세요: 010-1234-5678
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 3
[jmlee: 010-1234-5678]
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 1
이름을 입력하세요: kitae
전화번호를 입력하세요: 010-2345-6789
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 3
[jmlee: 010-1234-5678]
[kitae: 010-2345-6789]
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 2
이름을 입력하세요: jmlee
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 3
[kitae: 010-2345-6789]
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 4
pi@pi:~/prac/appendixA $
```

4. 문제 3을 수정해보자. '종료' 메뉴를 선택하며 현재 딕셔너리에 저장된 내용을 'phone_book.txt'에 저장하고, 프로그램을 실행을 시작할 때 'phone_book.txt' 파일에서 이름과 전화번호를 읽어 딕셔너리에 적재하도록 프로그램을 수정하라. 여기서는'phone_book.txt'가 다음과 같이 이미 있는 것으로 가정하라. 독자들의 라즈베리파이에 'phone_book.txt' 파일이 없다면 vi 등을 이용하여 임의로 작성해도 된다.

힌트 문자열을 ","로 분리하고자 하는 경우 문자열의 split() 함수를 이용하라.

```
pi@pi:~/prac/appendixA $ cat phone_book.txt
jmlee,010-1234-5678
kiate,010-2345-6789
pi@pi:~/prac/appendixA $
```

```
pi@pi:~/prac/appendixA $ python pracA-4.py
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 3
[jmlee: 010-1234-5678 ]
[kiate: 010-2345-6789]
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 1
이름을 입력하세요: ihjung
전화번호를 입력하세요: 010-3456-7890
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 3
[jmlee: 010-1234-5678 ]
[kiate: 010-2345-6789]
[ihjung: 010-3456-7890]
(1) 신규, (2) 삭제, (3) 조회, (4) 종료 >> 4
pi@pi:~/prac/appendixA $
```

```
pi@pi:~/prac/appendixA $ cat phone_book.txt
jmlee,010-1234-5678
kiate,010-2345-6789
ihjung,010-3456-7890
pi@pi:~/prac/appendixA $
```